

Predicción del Precio de Viviendas mediante Regresión Lineal Múltiple usando Machine Learning

Sebastián Osorio Arteaga
Instituto Tecnológico de Estudios Superiores Monterrey
Querétaro, México
Email: sebastian.oso03@gmail.com

Abstract—La valuación del precio de las viviendas es un proceso clave en el mercado inmobiliario que afecta directamente la toma de decisiones de compradores, vendedores o inversionistas. En este trabajo se implementa un modelo de regresión lineal múltiple, construido desde cero sin el uso de frameworks especializados de Machine Learning, con el objetivo de predecir el precio por unidad de área de viviendas a partir de características como la ubicación, la antigüedad y la accesibilidad a transporte público y servicios. Se utilizó el Real Estate Valuation Data Set, compuesto por 414 transacciones inmobiliarias del distrito de Sindian, en Nuevo Taipei, Taiwán. El proceso incluyó la extracción, transformación y limpieza de los datos (ETL), un análisis exploratorio para identificar relaciones entre las variables y el precio, el entrenamiento del modelo utilizando el algoritmo de gradiente descendiente con la función de costo de Mean Squared Error (MSE). El modelo fue evaluado mediante las métricas de MSE y el coeficiente de determinación R^2 , obteniendo resultados que evidencian tanto la capacidad predictiva del modelo como sus limitaciones.

I. INTRODUCCIÓN

La valuación del precio de una vivienda es un proceso fundamental dentro del mercado inmobiliario, ya que influye directamente en las decisiones de compradores, vendedores o inversionistas. Tradicionalmente, esta valuación se ha realizado mediante tasadores o expertos del sector, quienes estiman el precio de una propiedad con base en su experiencia y en la comparación con propiedades similares. Sin embargo, este enfoque tiene ciertas limitantes: ya que el proceso puede ser lento y, en muchos casos, representa un costo adicional para quien lo solicita.

Ante estas limitantes, han surgido enfoques basados en modelos estadísticos y de aprendizaje automático que buscan automatizar la estimación del precio de una vivienda a partir de sus características. Estos modelos permiten obtener predicciones de forma rápida y consistente, sin depender directamente del juicio de un experto, además de que pueden ser reentrenados conforme se actualizan las condiciones del mercado. Si bien existen enfoques más complejos para este tipo de problema, como redes neuronales convolucionales u otros modelos, en este trabajo se optó por implementar un modelo de regresión lineal desde cero, sin el uso de frameworks especializados de Machine Learning, con el objetivo de profundizar en la comprensión de los fundamentos matemáticos y algorítmicos detrás de este tipo de modelos [4].

En este trabajo se implementa un modelo de regresión lineal múltiple, entrenado mediante el algoritmo de gradiente descendiente, con el fin de predecir el precio por unidad de

área de viviendas ubicadas en el distrito de Sindian, en Nuevo Taipei, Taiwán. Para ello se realiza un proceso completo que abarca la extracción, transformación y carga (ETL) del dataset, un análisis exploratorio de los datos, la preparación de los mismos para el entrenamiento, la construcción del modelo y, finalmente, la evaluación de su desempeño mediante las métricas de Mean Squared Error (MSE) y el coeficiente de determinación (R^2).

II. DESCRIPCIÓN DEL DATASET

El dataset utilizado corresponde al Real Estate Valuation Data Set, disponible en el repositorio de Machine Learning de la UCI [1]. Contiene datos históricos de valuación de bienes raíces del distrito de Sindian, en Nuevo Taipei, Taiwán, recolectados entre agosto de 2012 y julio de 2013.

El conjunto de datos cuenta con 414 instancias y 7 variables en total: 6 variables predictoras y 1 variable objetivo.

- X1 transaction date: fecha de la transacción, en formato decimal (por ejemplo, 2013.250 = marzo 2013). Variable continua.
- X2 house age: antigüedad de la vivienda al momento de la transacción, en años.
- X3 distance to the nearest MRT station: distancia a la estación de metro (MRT) más cercana, en metros.
- X4 number of convenience stores: número de tiendas de conveniencia dentro del radio caminable de la vivienda.
- X5 latitude: coordenada geográfica de latitud, en grados.
- X6 longitude: coordenada geográfica de longitud, en grados.
- Y house price of unit area: Variable objetivo. Precio por unidad de área, en unidades de 10,000 nuevos dólares taiwaneses por Ping (1 Ping $\approx$ 3.3 m²).

III. EXTRACCIÓN, TRANSFORMACIÓN Y CARGA DE LOS DATOS (ETL)

El objetivo de esta etapa es preparar la información cruda para su análisis y uso posterior siguiendo las mejores prácticas de manejo de datos [3]. El proceso consistió en tres etapas principales: extracción desde la fuente original, transformación (limpieza, estandarización y generación de nuevas variables), y carga en un formato adecuado para su consumo.

A. Carga y limpieza inicial

El primer paso en esta sección es la carga del dataset, el dataset original se encontraba en formato Excel (.xlsx) y

después de cargar los datos, se verificó la ausencia de valores nulos.

B. Transformación de variables

Se realizaron las siguientes transformaciones:

- Eliminación de columna identificador: La columna "No" fue eliminada por ser un identificador único que no aporta información significativa para el modelo.
- Estandarización de nombres: Las columnas fueron renombradas eliminando los prefijos (X1, X2, etc.) para facilitar la lectura y manipulación del dataset.
- Transformación de fecha: La columna "transaction_date" se encontraba en formato decimal (e.g., 2012.916, donde $0.916 \approx 11/12$, indicando noviembre de 2012). Esta columna se dividió en dos variables separadas: "transaction_year" y "transaction_month", haciendo el dataset más legible y facilitando su análisis.

C. Exportación de datos procesados

Una vez finalizado el proceso de limpieza y transformación, el dataset se guardó en formato CSV, un formato más simple y universal que el Excel original, facilitando su carga en futuras etapas del proyecto.

IV. ANÁLISIS EXPLORATORIO DE LOS DATOS (EDA)

En esta etapa se busca identificar patrones, tendencias, relaciones entre variables, así como posibles anomalías o valores atípicos dentro del dataset, apoyándose principalmente de análisis estadístico y gráficas [3]. Esta etapa permite tomar decisiones informadas sobre qué variables incluir en el modelo o si es necesario transformar variables.

A. Distribución de variables

Para analizar la distribución de las variables del dataset, se generaron diagramas de caja y bigotes (boxplots) para cada una de ellas, como se muestra en la Figura 1.

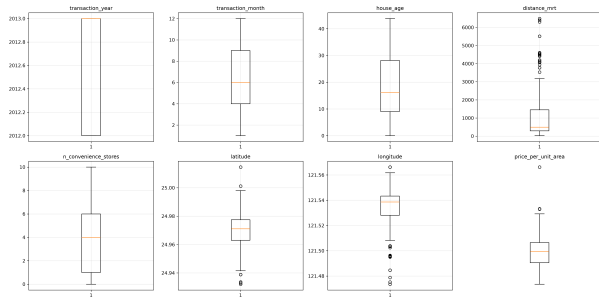


Fig. 1. Diagramas de caja y bigotes para todas las variables del dataset.

Como se observa en la Figura 1, la mayoría de las variables presentan distribuciones asimétricas y tienen rangos y unidades completamente diferentes, es importante tener esto en cuenta ya que se va a necesitar escalar los datos antes del entrenamiento del modelo para evitar que variables con rangos más grandes dominen el cálculo del gradiente.

B. Análisis de correlación

Para cuantificar las relaciones lineales entre las variables, se calculó la matriz de correlación de Pearson, mostrada en la Figura 2. El coeficiente de correlación de Pearson varía entre -1 y 1, donde valores cercanos a 1 indican correlación positiva fuerte, valores cercanos a -1 indican correlación negativa fuerte, y valores cercanos a 0 indican ausencia de correlación lineal [3].

El signo te dice la dirección si sube o baja con la variable y la correlación simplemente indica que se mueven juntos, no que si uno se mueve eso causa que se mueva el otro.

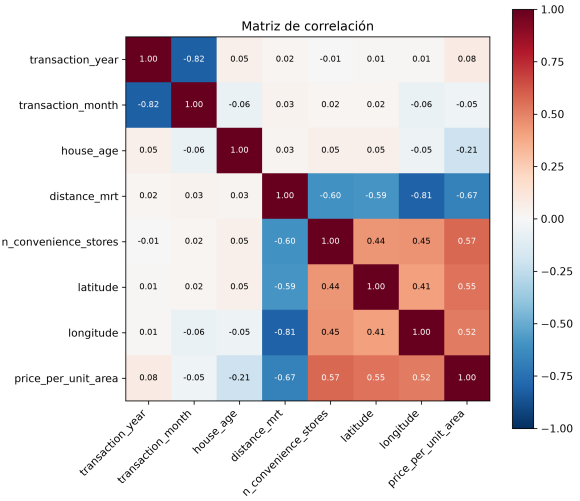


Fig. 2. Matriz de correlación entre todas las variables del dataset.

Analizando la Figura 2, se pueden identificar las siguientes relaciones con la variable objetivo price_per_unit_area:

- distance_mrt presenta una correlación negativa de -0.67, indicando una relación fuerte: a mayor distancia de la estación de metro, menor es el precio de la vivienda.
- n_convenience_stores muestra una correlación positiva de 0.57, sugiriendo que un mayor número de tiendas de conveniencia en la zona se asocia con precios más altos.
- latitude y longitude presentan correlaciones positivas de 0.55 y 0.52 respectivamente, indicando que la ubicación geográfica es un factor importante en la determinación del precio.
- house_age tiene una correlación negativa moderada de -0.21, sugiriendo que las viviendas más antiguas tienden a tener precios menores.
- transaction_year y transaction_month muestran correlaciones muy bajas (0.08 y -0.05 respectivamente), indicando que el momento de la transacción tiene poco impacto en el precio dentro del período analizado.

Es importante notar que distance_mrt, n_convenience_stores, latitude y longitude

presentan correlaciones moderadas entre sí, lo cual es esperable dado que todas están relacionadas con la ubicación de la propiedad. Esto sugiere que la ubicación de la vivienda es un factor determinante en la valuación de viviendas en Taipei.

C. Relación entre features y variable objetivo

Para visualizar de manera más detallada la relación entre cada variable predictora y el precio, se generaron scatter plots, mostrados en la Figura 3.

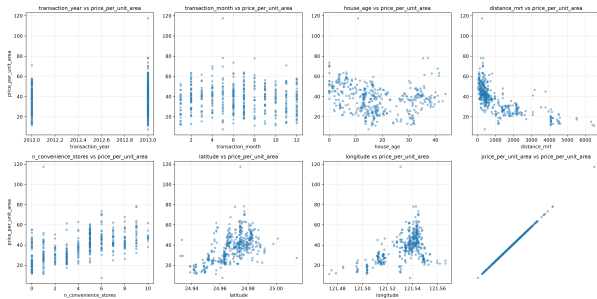


Fig. 3. Scatter plots de cada feature contra la variable objetivo (precio por unidad de área).

Los scatter plots confirman las observaciones de la matriz de correlación:

- `transaction_year` y `transaction_month` no muestran ninguna tendencia visual clara con el precio, consistente con sus bajas correlaciones.
- `house_age` muestra una tendencia negativa moderada con dispersión considerable, consistente con su correlación de -0.21.
- `n_convenience_stores`, `latitude` y `longitude` muestran tendencias positivas consistentes con sus coeficientes de correlación.

Un caso particular merece atención especial: `distance_mrt`. Aunque su correlación de -0.67 indica una relación fuerte, el scatter plot revela que dicha relación no es completamente lineal, sino que sigue una forma curva. El precio disminuye rápidamente a distancias cortas del metro y se estabiliza conforme aumenta la distancia. Dado que la regresión lineal asume una relación lineal entre las features y la variable objetivo, esta curva puede llegar a afectar nuestro modelo, así que se tendría que transformar de alguna manera para hacer esta relación entre `distance_mrt` y la variable objetivo más lineal.

Otro patrón interesante se observa en las gráficas de `latitude` y `longitude`, donde se aprecia una agrupación de precios altos en una zona específica. Esto podría corresponder al centro de la ciudad o una zona particularmente céntrica de Nuevo Taipei, lo que reforzaría la importancia de la ubicación en la determinación del precio.

D. Selección de características

Se decidió por conservar todas las variables del dataset, ya que en el contexto en el que estamos de bienes raíces y

avalúos de casas, todas estas variables aportan información importante. Factores de ubicación y accesibilidad como `distance_mrt`, `n_convenience_stores`, `latitude` y `longitude` muestran la centralidad y la proximidad a servicios y transporte público, son variables conocidas por todos que son determinantes al poner precio a una vivienda. Por otro lado, `transaction_year` y `transaction_month` nos pueden hablar de la inflación y las condiciones del mercado inmobiliario en ese momento, mientras que `house_age` impacta directamente en el valor percibido de la propiedad, ya que una vivienda nueva generalmente tiene un precio mayor que una de características similares pero con mayor antigüedad.

Adicionalmente, se evaluó empíricamente el desempeño del modelo con y sin estas variables, observando que el R^2 fue mayor incluyéndolas todas. Esto sugiere que aunque algunas variables como `latitude` y `longitude` podrían compartir información con `distance_mrt`, también aportan información espacial adicional que el modelo aprovecha para mejorar las predicciones.

E. Transformación de variables

La única variable que requirió transformación fue `distance_mrt`. Como se mencionó anteriormente, el scatter plot reveló que su relación con el precio no es lineal sino que sigue una forma curva. Para corregir esto, se aplicó una transformación logarítmica:

$$\log_distance_mrt = \log(distance_mrt)$$

El logaritmo convierte la relación curva en una relación más lineal, ajustándose mejor a los supuestos del modelo de regresión lineal. Para verificar la efectividad de esta transformación, se compararon las correlaciones:

- `distance_mrt`: correlación de -0.67
- `log_distance_mrt`: correlación de -0.73

El aumento en la magnitud de la correlación confirma que la transformación logarítmica captura mejor la relación entre la distancia a la estación y el precio de la propiedad. La Figura 4 muestra visualmente esta mejora.

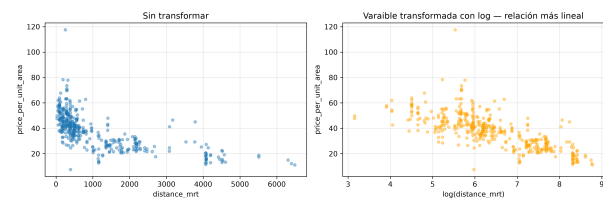


Fig. 4. Comparación de la relación entre distancia al metro y precio: (a) variable original, (b) variable transformada con logaritmo.

Por lo tanto, se utilizó `log_distance_mrt` en lugar de `distance_mrt` como variable predictora en el modelo.

V. PREPARACIÓN DE DATOS PARA EL MODELO

A. División train/test

Una vez procesado el dataset, el siguiente paso fue dividirlo en dos conjuntos: entrenamiento (train) y prueba (test). Esta división es fundamental para evaluar la capacidad del modelo de generalizar sobre datos que no fueron vistos durante el entrenamiento [5].

Antes de realizar la división, se aplicó un *shuffle* (mezcla aleatoria) al dataset para eliminar cualquier sesgo que pudiera existir por el orden original de las instancias. Se asignó el 80% de las instancias al conjunto de entrenamiento y el 20% restante al conjunto de prueba, siguiendo la regla 80-20. Esta proporción es comúnmente utilizada ya que destina una mayor cantidad de datos al entrenamiento para garantizar una estimación de parámetros más robusta, mientras reserva una porción representativa para la evaluación.

B. Agregación del término de bias

En regresión lineal, el modelo busca encontrar una función de la forma:

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

donde θ_0 representa el término de bias. Para incorporar este término, se añadió una columna de unos al inicio de cada muestra, tanto en el conjunto de entrenamiento como en el de prueba. Esto permite que el parámetro θ_0 se trate como cualquier otro parámetro del modelo y sea actualizado directamente por el algoritmo de gradiente descendiente, sin requerir un tratamiento especial.

C. Estandarización con Z-score

Como se observó en el EDA, las variables del dataset se encuentran en rangos y unidades completamente diferentes. Por ejemplo, `distance_mrt` su unidad es en metros con valores de hasta 6,500, mientras que `transaction_year` varía entre 2012 y 2013 únicamente. Es necesario escalar los datos antes del entrenamiento, ya que sin estandarización, las features con rangos más grandes dominarían el cálculo del gradiente, dificultando la convergencia del modelo.

Se utilizó la estandarización Z-score, cuya fórmula es:

$$z_i = \frac{x_i - \bar{x}}{\sigma_x}$$

donde $\bar{x}$ es la media aritmética de x y σ_x su desviación estándar. El resultado es que cada variable queda con media de cero y desviación estándar uno, haciendo que todas las features contribuyan de forma proporcional al entrenamiento.

Un aspecto crítico de la implementación es que los parámetros $\bar{x}$ y σ_x se calcularon únicamente sobre el conjunto de entrenamiento y posteriormente se aplicaron al conjunto de prueba con los mismos valores. Esto evita el *data leakage*, que ocurre cuando información del conjunto de prueba influye en el proceso de entrenamiento, lo cual produciría métricas de evaluación artificialmente optimistas.

VI. CONSTRUCCIÓN DEL MODELO

A. Inicialización de parámetros

Los parámetros del modelo se inicializaron con ceros. El vector de parámetros tiene una dimensión por cada feature más el término de sesgo, resultando en un total de 8 parámetros.

B. Función de hipótesis

La función de hipótesis es la función que va a usar el modelo para predecir valores de la variable objetivo, para este problema se va a usar una función para regresión lineal que tiene la siguiente forma:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

donde θ_0 es el término de sesgo y $\theta_1, \dots, \theta_n$ son los parámetros asociados a cada feature. En la implementación, esta función (denominada h) recorre cada parámetro y lo multiplica por el valor correspondiente de la muestra, acumulando la suma para obtener la predicción final.

C. Función de costo: MSE

Para medir el error del modelo se utilizó el Mean Squared Error (MSE), que calcula el promedio de los errores al cuadrado entre las predicciones del modelo y los valores reales [2]:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

donde n es el número de muestras, Y_i es el valor real de la muestra i y $\hat{Y}_i$ es el valor predicho por el modelo. Elevar al cuadrado el error tiene dos propósitos fundamentales: (1) penaliza errores grandes de forma más severa que errores pequeños, y (2) elimina los signos negativos para que los errores no se cancelen entre sí. En la implementación, el MSE se calcula al final de cada época tanto para el conjunto de train como para el de test.

D. Función de optimización: Gradiente descendiente

El gradiente descendiente es un algoritmo de optimización que ajusta los parámetros del modelo de forma iterativa con el objetivo de minimizar la función de costo [4]. En cada iteración, se calcula la derivada parcial del error con respecto a cada parámetro para obtener el gradiente, un vector que indica la dirección de la pendiente de la función de costo. Moviendo los parámetros en la dirección contraria al gradiente en cada iteración, el modelo va reduciendo su error progresivamente hasta converger a un mínimo.

La regla de actualización para cada parámetro θ_j es:

$$\theta_j := \theta_j - \frac{\alpha}{m} \sum_{i=1}^m [(h_{\theta}(x_i) - y_i)x_i]$$

Donde m es el número de muestras, $h_{\theta}(x_i)$ la predicción del modelo, y_i el valor real, y α es la tasa de aprendizaje (*learning rate*). El learning rate es un hiperparámetro que controla qué tan grandes son los pasos que da el algoritmo en

cada iteración. A diferencia de los parámetros del modelo, que son aprendidos automáticamente durante el entrenamiento, los hiperparámetros son definidos manualmente antes de entrenar. Un α muy grande puede provocar que el algoritmo diverja, mientras que uno muy pequeño hace que converja demasiado lento [7]. Para este modelo se definió $\alpha = 0.1$, un valor que balancea adecuadamente la velocidad de convergencia con la estabilidad del entrenamiento.

VII. EVALUACIÓN DEL MODELO

A. Métricas MSE y R^2

Para evaluar el desempeño del modelo se utilizaron dos métricas: el MSE descrito anteriormente y el coeficiente de determinación R^2 , cuya fórmula es [3]:

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

donde SS_{RES} es la suma de cuadrados de los residuos (diferencia entre valores reales y predichos), SS_{TOT} es la suma total de cuadrados (diferencia entre valores reales y la media), y_i es el valor real, $\hat{y}_i$ es el valor predicho, y $\bar{y}$ es la media de los valores reales.

El R^2 indica qué proporción de la variabilidad de la variable objetivo es explicada por el modelo. Un valor de $R^2 = 1$ indica que el modelo explica perfectamente los datos, mientras que $R^2 = 0$ indica que el modelo no es mejor que simplemente predecir la media. Valores negativos son posibles cuando el modelo es peor que predecir la media.

B. Curva de pérdida train vs test

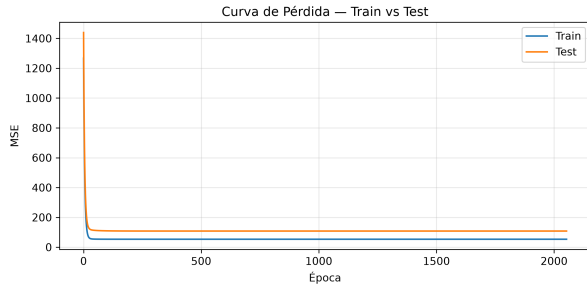


Fig. 5. Evolución del MSE durante el entrenamiento para los conjuntos de train y test.

La Figura 5 muestra la evolución del MSE a lo largo de las épocas de entrenamiento. Ambas curvas parten de un MSE elevado cercano a 1,400 y descienden rápidamente durante las primeras épocas, estabilizándose alrededor de la época 500. El modelo convergió en 2,054 épocas, antes de alcanzar el límite de 60,000 épocas establecido.

Los resultados finales del modelo fueron:

- Train: MSE = 53.48, $R^2 = 0.6830$
- Test: MSE = 108.32, $R^2 = 0.5592$

El modelo logró explicar el 68.3% de la varianza en el conjunto de entrenamiento y el 55.92% en el conjunto de prueba. La diferencia entre ambas métricas indica que el

modelo tiene cierta capacidad de generalización, aunque existe una brecha notable entre el desempeño en train y test.

C. Análisis de underfitting y overfitting

Analizando la curva de pérdida y las métricas obtenidas, se pueden identificar dos fenómenos importantes:

Overfitting: La curva de pérdida del conjunto de entrenamiento converge a un MSE notablemente menor (53.48) que la curva del conjunto de prueba (108.32), lo cual indica que el modelo se ajustó mejor a los datos de entrenamiento que a los datos nuevos. Esto se aprecia visualmente en la Figura 5, donde la curva de test se mantiene consistentemente por encima de la curva de train a lo largo de todo el entrenamiento. La diferencia en R^2 (0.6830 vs 0.5592) también confirma este comportamiento.

Underfitting: El valor de $R^2 = 0.5592$ en el conjunto de prueba sugiere que aproximadamente el 44% de la variabilidad del precio no es explicada por el modelo. Esto es una limitación inherente de la regresión lineal: al ser un modelo de baja complejidad, asume que las relaciones entre las features y la variable objetivo son lineales, lo cual puede no capturar toda la estructura presente en los datos. Como se observó en el EDA, algunas relaciones (como la de `distance_mrt` antes de la transformación logarítmica) son no lineales, y aunque se aplicó la transformación, es posible que existan otras relaciones no lineales que el modelo no pueda capturar [6].

Para mitigar estos problemas, se podrían considerar las siguientes estrategias:

- Modelos más complejos: Explorar modelos como Random Forest, Gradient Boosting o redes neuronales que puedan capturar relaciones no lineales.
- Más datos: Aumentar el tamaño del dataset podría ayudar a mejorar la generalización.

D. Predicciones vs valores reales

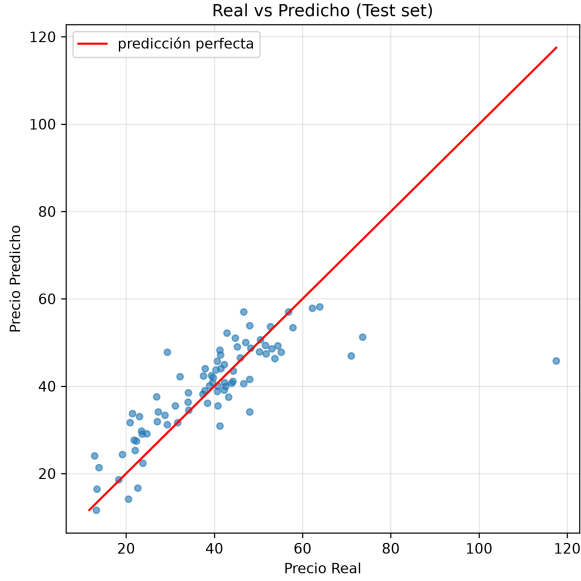


Fig. 6. Scatter plot de predicciones vs valores reales en el conjunto de test. La línea roja representa la predicción.

La Figura 6 muestra un scatter plot de las predicciones del modelo contra los valores reales en el conjunto de test. La línea roja diagonal representa la predicción perfecta (donde predicción = valor real).

Se pueden observar los siguientes patrones:

- Dispersión alrededor de la línea: Los puntos se distribuyen alrededor de la línea de predicción, pero con una dispersión considerable, lo cual es consistente con el $R^2 = 0.5592$ obtenido.
- Bias y varianza: La dispersión de los puntos indica la varianza del error del modelo. Aunque no hay un sesgo sistemático evidente, la magnitud de la dispersión sugiere que el modelo tiene limitaciones para predecir con precisión en todos los rangos de precios.

E. Interpretación de parámetros

Los parámetros finales del modelo después del entrenamiento fueron:

$$\theta = [37.62, 2.72, 1.03, -2.40, -6.92, 1.46, 3.19, 0.59]$$

donde cada parámetro corresponde a:

- $\theta_0 = 37.62$: Bias o término de intercepción
- $\theta_1 = 2.72$: transaction_year
- $\theta_2 = 1.03$: transaction_month
- $\theta_3 = -2.40$: house_age
- $\theta_4 = -6.92$: log_distance_mrt
- $\theta_5 = 1.46$: n_convenience_stores
- $\theta_6 = 3.19$: latitude
- $\theta_7 = 0.59$: longitude

Es importante notar que estos parámetros están en la escala estandarizada (Z-score), por lo que su magnitud relativa nos indica la importancia de cada feature:

- log_distance_mrt ($\theta_4 = -6.92$) tiene el coeficiente más grande en magnitud, confirmando que es la variable más importante para predecir el precio. El signo negativo indica que a mayor distancia del metro, menor es el precio, lo cual es consistente con la correlación negativa observada en el EDA.
- latitude ($\theta_6 = 3.19$) y n_convenience_stores ($\theta_5 = 1.46$) tienen coeficientes positivos, indicando que ubicaciones con mayor latitud y más tiendas de conveniencia tienden a tener precios más altos.
- house_age ($\theta_3 = -2.40$) tiene un coeficiente negativo, lo que indica que las casas más antiguas tienden a tener menor precio, aunque su magnitud es menor que la de log_distance_mrt.
- transaction_year y transaction_month tienen coeficientes relativamente pequeños, lo cual es consistente con las bajas correlaciones observadas en el EDA.

VIII. CONCLUSIONES

En este trabajo se implementó un modelo de regresión lineal múltiple desde cero para predecir el precio por unidad de área de viviendas en el distrito de Sindian, Nuevo Taipei, Taiwán. El proceso completo incluyó:

- 1) ETL: Extracción, transformación y carga de datos, incluyendo la limpieza del dataset, renombrado de columnas, y transformación de la fecha de transacción de formato decimal a año y mes separados.
- 2) EDA: Análisis exploratorio que reveló relaciones importantes entre las variables, identificando que distance_mrt, n_convenience_stores, latitude y longitude son las variables más correlacionadas con el precio. Se detectó una relación no lineal entre distance_mrt y el precio, lo que motivó la aplicación de una transformación logarítmica.
- 3) Preparación de datos: División 80-20 en train/test, adición del término de sesgo, y estandarización Z-score para asegurar que todas las features contribuyan proporcionalmente al entrenamiento.
- 4) Implementación del modelo: Construcción desde cero con función de hipótesis, función de costo (MSE), y algoritmo de optimización (gradiente descendiente) con learning rate $\alpha = 0.1$.
- 5) Evaluación: El modelo alcanzó un $R^2 = 0.6830$ en train y $R^2 = 0.5592$ en test, explicando aproximadamente el 56% de la varianza en datos no vistos.

Los resultados demuestran que la regresión lineal es capaz de capturar relaciones significativas entre las características de las viviendas y su precio, particularmente la distancia a la estación de metro, que resultó ser la variable más importante. Sin embargo, las limitaciones del modelo (underfitting parcial y overfitting moderado) sugieren que existen relaciones no

lineales y patrones más complejos en los datos que un modelo lineal simple no puede capturar completamente.

Para poder explicar de mejor manera la varianza de los datos, es decir una R^2 más alta, se podría recomendar hacer lo siguiente:

- Uso de modelos no lineales y frameworks especializados: Se sugiere explorar modelos más complejos, como lo puede ser Random Forest o Redes Neuronales. Al ser modelos más complejos que una regresión lineal, aunque su implementación desde cero resulta poco práctica, por lo que se recomienda utilizar frameworks de Machine Learning como: Scikit-Learn, TensorFlow o PyTorch. Estas herramientas no solo simplifican el desarrollo, sino que aprovechan implementaciones algorítmicas altamente optimizadas.

Este proyecto demuestra la importancia de comprender tanto las fortalezas como las limitaciones de los modelos de Machine Learning, y cómo un análisis exploratorio adecuado puede guiar decisiones importantes como la transformación de variables y la selección de features.

REFERENCIAS

REFERENCES

- [1] Yeh, I. (2018). *Real Estate Valuation Data Set*. UCI Machine Learning Repository. [Online]. Available: <https://archive.ics.uci.edu/dataset/477/real+estate+valuation+data+set>
- [2] N. Donges, "Gradient Descent in Machine Learning: A Basic Introduction," *Built In*, Aug. 1, 2024. [Online]. Available: <https://builtin.com/data-science/gradient-descent>
- [3] GeeksforGeeks, "Exploratory Data Analysis," [Online]. Available: <https://www.geeksforgeeks.org/data-analysis/what-is-exploratory-data-analysis/>
- [4] GeeksforGeeks, "Gradient Descent Algorithm in Machine Learning," [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/gradient-descent-algorithm-and-its-variants/>
- [5] GeeksforGeeks, "How To Do Train Test Split Using Sklearn In Python," [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/how-to-do-train-test-split-using-sklearn-in-python/>
- [6] GeeksforGeeks, "Underfitting and Overfitting in ML," [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/>
- [7] GeeksforGeeks, "Learning Rate in Neural Network," [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/impact-of-learning-rate-on-a-model/>